
课程结构

课程大致分为四块：

1. Deep Learning Basics

- 图像分类
- KNN 与线性分类器
- 损失函数
- 优化
- 反向传播
- 多层感知机与神经网络

2. Perceiving and Understanding the Visual World

- CNN
- 经典 CNN 架构
- RNN
- Attention / Transformer
- 目标检测
- 图像分割
- 视频理解
- 视觉语言

3. Reconstructing and Interacting with the Visual World

- 自监督学习
- 生成模型
- 3D 视觉
- 机器人学习

4. Human-Centered AI

- 公平性
- 偏见
- 伦理风险
- 社会影响

视觉与深度学习的发展

生物视觉启发

- **Hubel & Wiesel, 1959**: 发现视觉皮层中存在对特定方向、边缘、运动响应的神经元。
- 简单细胞 **Simple Cells**: 响应局部方向、边缘等模式。
- 复杂细胞 **Complex Cells**: 对位置变化更稳健，体现一定的平移不变性。

这一脉络后来影响了卷积神经网络中的：

- 局部感受野
- 分层特征

- 卷积
- 池化
- 对平移和局部扰动的鲁棒性

早期模型驱动

早期计算机视觉试图手工定义视觉理解流程：

- **Roberts, 1963**：从 2D 图像推断 3D 几何结构。
- **David Marr, 1970s**：提出视觉表示阶段：
 - 原始图像
 - Primal Sketch：边缘、线段、纹理等低层结构
 - 2.5D Sketch：局部表面方向、深度关系
 - 3D Model：物体级三维表示
- **Recognition by Parts**：用部件组合识别物体，例如 generalized cylinders。
- **Canny Edge Detector, 1986**：经典边缘检测。
- **Normalized Cuts, 1997**：图像分割中的图划分思想。
- **SIFT, 1999**：局部特征匹配，具有尺度和旋转鲁棒性。
- **Viola-Jones, 2001**：机器学习驱动的人脸检测，是早期成功的视觉应用之一。

这一阶段的特点：

- 强依赖手工特征与几何假设。
- 能解决部分受控问题，但难以覆盖真实世界复杂变化。

神经网络发展

- **Perceptron, 1958**：早期线性分类器。
- **Minsky & Papert, 1969**：指出单层感知机无法学习 XOR，引发神经网络低潮。
- **Neocognitron, 1980**：受视觉皮层启发，交替使用类似卷积和池化的结构，但缺少有效训练算法。
- **Backpropagation, 1986**：反向传播可高效计算多层网络梯度。
- **LeNet, 1998**：卷积网络成功应用于手写数字识别。
- **2000s Deep Learning**：更深网络开始被探索，但受限于数据、算力和训练技术。

ImageNet 与 AlexNet

深度学习真正爆发来自三个条件同时成熟：

- **Data**：ImageNet 提供大规模标注图像，分类挑战包含约 1000 类、百万级图像。
- **Algorithms**：反向传播、卷积网络、激活函数、正则化、优化技巧持续积累。
- **Computation**：GPU 和后来的 Tensor Cores 让大模型训练可行。

关键节点：

- **ImageNet, 2009**：大规模视觉识别数据集。
- **AlexNet, 2012**：在 ImageNet 上显著超过传统方法，使深度学习成为计算机视觉主流。

深度学习在视觉中的应用扩展

从 2012 年以后，深度学习快速扩展到多个视觉任务：

- 图像分类 **Image Classification**：判断整张图像属于哪个类别。
 - 图像检索 **Image Retrieval**：根据视觉相似性搜索图像。
 - 目标检测 **Object Detection**：输出物体类别和位置框。
 - 语义分割 **Semantic Segmentation**：为每个像素预测语义类别。
 - 实例分割 **Instance Segmentation**：区分同类不同实例。
 - 视频理解 **Video Understanding**：识别动作、事件和时序结构。
 - 姿态估计 **Pose Estimation**：识别人或物体关键点。
 - 图像描述 **Image Captioning**：把图像转换为自然语言描述。
 - 视觉关系 **Visual Relationship**：理解对象之间的关系，例如 “person riding bike”。
 - 风格迁移 **Style Transfer**：分离并重组内容和风格。
 - 生成模型 **Generative Models**：GAN、扩散模型、文本生成图像。
 - 视觉语言模型 **Vision-Language Models**：例如 CLIP，将图像和文本映射到共享语义空间。
 - 3D 视觉 **3D Vision**：从 2D/多视角数据恢复三维结构。
 - 具身智能 **Embodied AI / Robotics**：视觉感知与动作决策结合。
-

模型类型

- **MLP / 全连接网络**：把输入展平后用矩阵乘法建模，缺少显式空间归纳偏置。
 - **CNN**：利用局部连接、权重共享、层级特征，适合图像网格结构。
 - **RNN**：处理序列数据，适合早期视频、语言、序列建模。
 - **Attention / Transformer**：用注意力机制建模长程依赖，成为视觉语言和大模型的重要基础。
 - 生成模型：学习数据分布，可以生成图像、补全图像、跨模态生成。
-

第 2 讲：分类器

图像分类任务

图像分类 **Image Classification**：

给定一张图像，从预设类别集合中预测一个标签。

例子：

输入：一张 $32 \times 32 \times 3$ 的 CIFAR-10 图像

输出：{airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck} 中的一个类别

计算机看到的图像是张量：

$$x \in \mathbb{R}^{H \times W \times C}$$

其中像素值通常在 $[0, 255]$ 或归一化后的 $[0, 1]$ ，数据张量为分辨率 $800 \times 200 \times 3$ 通道。

视觉分类的典型挑战：视角改变导致像素大幅变化、- **Illumination**：光照变化改变颜色和阴影、背景干扰、目标被遮挡、类间差异等。

数据驱动方法

传统手写规则难以穷尽所有视觉变化，因此采用数据驱动方法：

1. 收集带标签的数据集。
2. 用机器学习算法训练分类器。
3. 在未见过的新图像上评估。

标准流程：

训练集 train：学习模型参数

验证集 validation：选择超参数

测试集 test：最终评估泛化性能，只在最后使用一次

1. Nearest Neighbor 分类器

1.1 基本思想

最近邻分类器不真正“训练”，而是记住所有训练样本。

预测时：

1. 计算测试图像与每张训练图像的距离。
2. 找到最相似的训练图像。
3. 复制它的标签作为预测结果。

复杂度：

阶段	复杂度	评价
训练	$O(1)$	只是记忆数据
预测	$O(N)$	每次预测都要和训练集比较，速度慢

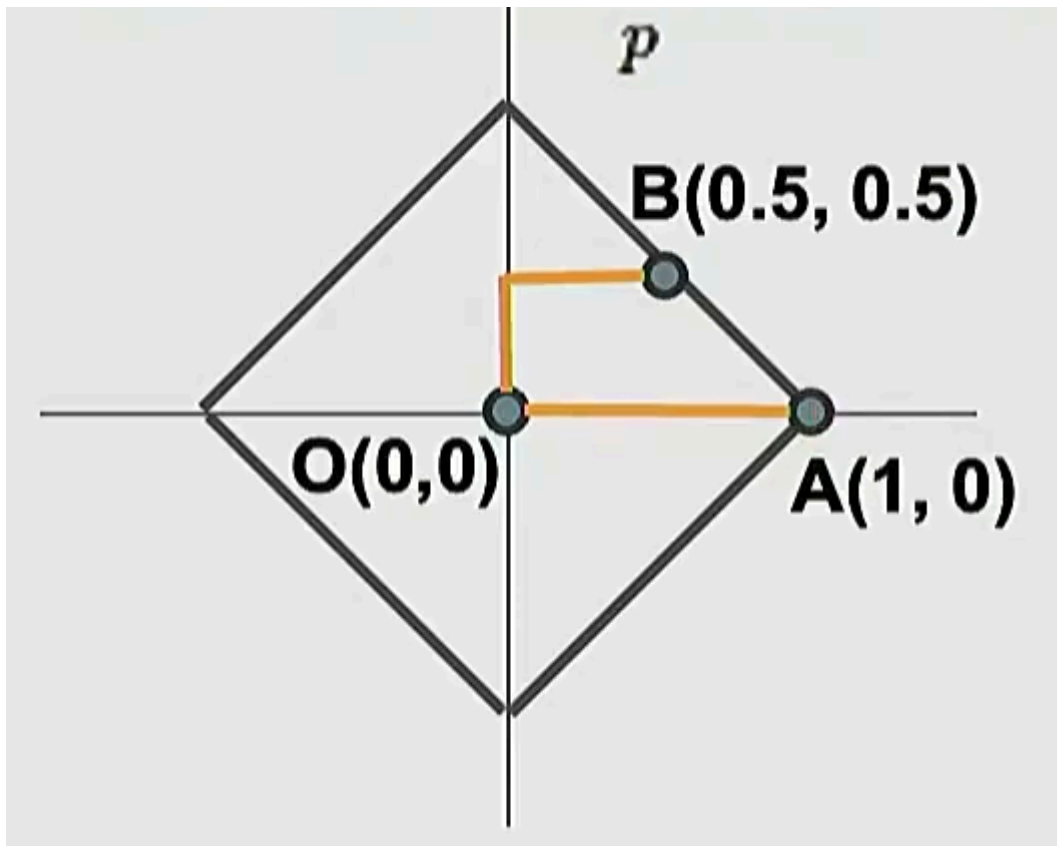
在实际深度学习系统中，通常希望训练可以慢一些，但预测要快。

1.2 距离度量

常见距离：

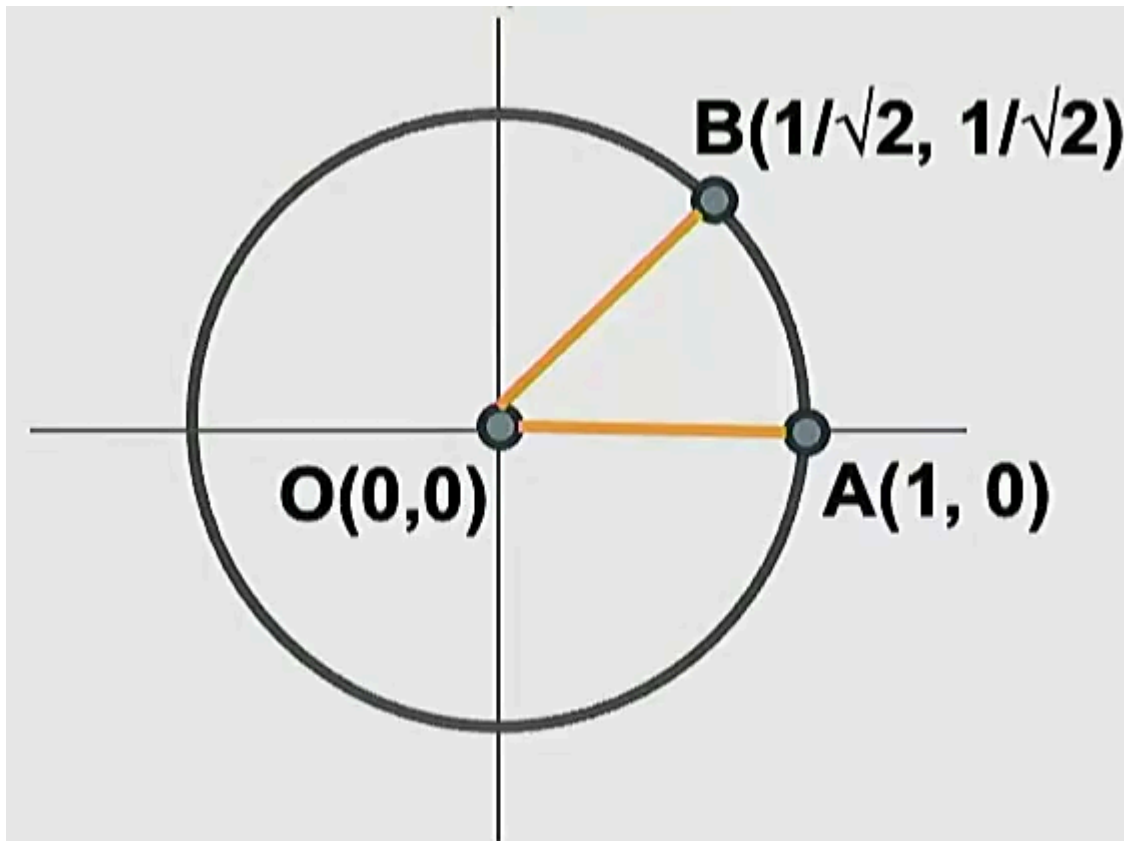
L1 / Manhattan Distance

$$d_1(x, y) = \sum_i |x_i - y_i|$$



L2 / Euclidean Distance

$$d_2(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$$



二者差异：

- L1 更像沿网格线移动，适合想做特征保留。
 - L2 是直线距离。
 - 距离选择会改变决策边界。
-

2. K-Nearest Neighbors

KNN 是最近邻的推广：

- 不只看最近的 1 个样本。
- 找到最近的 K 个样本。
- 用多数投票决定类别。

关键超参数：

- K 的取值
- 距离函数，例如 L1 或 L2

直接比较像素距离通常不是可靠的视觉语义度量。

例如：

- 图像平移 1 个像素，语义不变，但像素距离可能变大。

- 加一点色调变化，语义不变，但像素距离变化明显。
- 遮挡一部分图像，像素距离可能与完全不同语义的图像相近。

所以 KNN 是理解数据驱动分类的好起点，但不是现代图像分类的主力方法。

超参数选择

超参数 **Hyperparameters** 是算法本身的选择，不是训练过程中直接学出来的参数。

例如：

- KNN 中的 K
- 距离度量 L1 / L2
- 学习率
- 正则化强度

错误做法：

- 在训练集上选超参数：容易过拟合。
- 在测试集上反复选超参数：会污染测试集，导致泛化评估失真。

正确做法：

train: 训练不同模型

validation: 选择超参数

test: 只在最终模型确定后评估一次

小数据集可以使用 **Cross-Validation** 交叉验证：

- 把训练数据分成多个 fold。
- 每次用一个 fold 做验证集，其余做训练集。
- 对多次验证结果取平均。

深度学习中交叉验证不常用，原因是训练成本高，通常使用固定验证集。

3. 线性分类器：Parametric Approach

KNN 需要保留整个训练集，而线性分类器只保留参数。

线性分类器对输入做矩阵乘法：

$$f(x, W, b) = Wx + b$$

其中：

- x ：输入图像展平后的向量。
- W ：权重矩阵。
- b ：偏置向量。
- $f(x, W, b)$ ：每个类别的分数，也叫 logits。

例：以 CIFAR-10 将 image 分类到 10 个类中

$$x \in \mathbb{R}^{3072}$$

因为：

$$32 \times 32 \times 3 = 3072$$

若有 10 个类别：

$$W \in \mathbb{R}^{10 \times 3072}, \quad b \in \mathbb{R}^{10}$$

输出：

$$s = Wx + b \in \mathbb{R}^{10}$$

s 为 10×1 的矩阵，每个 s_j 表示图像属于第 j 类的分数。

- 损失函数 **Loss Function**

- 衡量当前参数 W, b 的预测有多差。

- 优化算法 **Optimization**

- 找到让损失尽可能小的参数。

数据集平均损失：

$$L = \frac{1}{N} \sum_{i=1}^N L_i$$

其中 L_i 是第 i 个样本的损失。

4. Softmax 分类器

它把 logits 转换为概率：

$$p_j = \frac{e^{s_j}}{\sum_k e^{s_k}}$$

其中：

- s_j ：第 j 类的 logit。

- p_j ：第 j 类的预测概率。正确类别为 y_i 时，单样本交叉熵损失：

$$L_i = -\log p_{y_i}$$

等价于：

$$L_i = -s_{y_i} + \log \sum_j e^{s_j}$$

- 正确类别概率越接近 1，损失越接近 0。

- 正确类别概率越接近 0，损失越大。

初始化时如果所有类别分数接近相等，且共有 C 类：

$$p_j \approx \frac{1}{C}$$

所以：

$$L_i \approx -\log \frac{1}{C} = \log C$$

例如 CIFAR-10 有 10 类：

$$L_i \approx \log 10 \approx 2.3$$

附加：**Multiclass SVM Loss**

SVM 损失关注的是 **margin**：正确类别分数应该比错误类别至少高出一个间隔 Δ 。

单样本多分类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

通常取：

$$\Delta = 1$$

含义：

- 如果错误类别分数 s_j 太接近或超过正确类别分数 s_{y_i} ，产生损失。
- 如果正确类别分数已经比错误类别高至少 Δ ，该错误类别贡献 0 损失。

当所有初始分数都接近 0 时：

$$L_i \approx C - 1$$

因为除正确类别外，每个错误类别都会贡献约 1 的 margin loss。

对比 **Softmax vs SVM**

对比点	Softmax	Multiclass SVM
目标	最大化正确类别概率	让正确类别分数超过错误类别至少一个 margin
输出解释	概率分布	类别分数间隔
损失形式	交叉熵	hinge loss
对分数变化的敏感度	即使分类正确，仍会继续推动概率更接近 1	margin 满足后损失为 0
常见用途	深度学习分类主流	传统线性分类与理解 margin 很重要

重要区别：

- Softmax 关心完整概率分布。
 - SVM 更关心正确类别和错误类别之间是否拉开足够间隔。
 - 两者都需要通过优化算法调整 W ，本讲主要先定义损失，后续讲优化。
-

第 3 讲：正则化和优化

1. 从分类器到完整的目标函数

第 2 讲已经建立了图像分类的基本框架：

- 数据集： (x_i, y_i) ，其中 x_i 是图像， y_i 是类别标签。
- 打分函数： $f(x, W)$ ，线性分类器中常写作 $f(x, W) = Wx + b$ 。
- 损失函数：衡量当前参数产生的分数有多差。

训练时真正最小化的是 数据损失 + 正则化损失：

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i) + \lambda R(W)$$

其中：

- $\frac{1}{N} \sum_i L_i$ ：要求模型预测匹配训练数据。
 - $R(W)$ ：对参数形态施加偏好。
 - λ ：正则化强度，是超参数。
-

2. 正则化

正则化会阻止模型把训练集拟合得过于完美，所以训练损失通常会变大；但它减少了模型对训练噪声、偶然模式和异常样本的依赖，因此测试集泛化可能更好。

正则化的核心目标：

- 偏好更简单的模型。
- 避免过拟合训练数据中的噪声。
- 在多个能解释训练数据的假设中，选择更可能泛化的一个。
- 有时还能改善优化曲面的性质，例如增加曲率，让优化更稳定。

这对应 Occam's Razor：

在多个竞争解释中，优先选择更简单的解释。

2.1 常见正则化形式

L2 Regularization

$$R(W) = \sum_k W_k^2$$

特点：

- 惩罚大权重。
- 倾向于把权重“摊开”，让多个特征共同承担预测。
- 在深度学习里常和 weight decay 相关。

L1 Regularization

$$R(W) = \sum_k |W_k|$$

特点：

- 更容易产生稀疏权重。
- 倾向于让部分参数变成 0，从而做特征选择。

Elastic Net

$$R(W) = \alpha \sum_k |W_k| + \beta \sum_k W_k^2$$

结合 L1 的稀疏性和 L2 的平滑惩罚。

更复杂的正则化

- Dropout
- Batch Normalization
- Stochastic Depth
- Fractional Pooling
- Data Augmentation

这些方法不一定都以显式 $R(W)$ 的形式出现，但本质上都在限制模型过拟合或改善训练动态。

3. 优化问题：如何找到好的 W

3.1 Gradient

任意方向的斜率可以看作该方向与梯度的点积；最陡下降方向是负梯度方向。

数值梯度用有限差分近似：

$$\frac{\partial L}{\partial W_k} \approx \frac{L(W + he_k) - L(W)}{h}$$

用解析梯度训练，用数值梯度做 gradient check。

3.2 梯度下降

基础梯度下降更新：

$$W \leftarrow W - \eta \nabla_W L(W)$$

其中：

- η : learning rate, 学习率/步长。
- $\nabla_W L(W)$: 当前参数处的梯度。

学习率太大可能导致 loss 爆炸；学习率太小会收敛很慢。

4. 随机梯度下降:Stochastic Gradient Descent

完整数据损失：

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i$$

当 N 很大时，每一步都遍历全量数据非常昂贵。SGD 用数据子集进行操作：

$$\nabla_W L(W) \approx \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_W L_i$$

常见 batch size: 32/64/128/256

SGD 的核心取舍：

- 每一步更便宜。
 - 梯度有噪声。
 - 噪声有时能帮助逃离鞍点，但也会造成震荡。
-

4.1 SGD 的三个典型问题

Problem 1: 病态曲率 损失在一个方向变化很快、另一个方向变化很慢：

- 陡峭方向上会来回震荡。
- 平缓方向上前进很慢。

Problem 2: 局部最小值与鞍点

- 鞍点附近梯度可能接近 0，普通梯度下降会变慢甚至卡住。

Problem 3: Mini-batch 噪声

- SGD 用小批量估计梯度，所以每一步方向不一定等于真实全量梯度，会产生抖动。
-

4.2 Momentum

SGD + Momentum 引入速度变量，保留历史方向：

$$v \leftarrow \rho v - \eta \nabla_W L(W)$$

按速度更新

$$W \leftarrow W + v$$

其中：

- ρ : momentum / friction, 常见取值 0.9 或 0.99。
 - 在连续一致的方向上加速。
 - 在震荡方向上互相抵消。
 - 更容易穿过局部噪声和鞍点附近的平坦区域。
-

5. RMSProp

RMSProp 为每个参数维护梯度平方的滑动平均：

$$square \leftarrow decay \cdot square + (1 - decay) \cdot g^2$$

$$W \leftarrow W - \eta \frac{g}{\sqrt{square} + \epsilon}$$

作用：

- 梯度长期很大的方向，步长被压小。
 - 梯度长期很小的方向，步长相对变大。
 - 可以看作 per-parameter learning rate / adaptive learning rate。
-

6. Adam 与 AdamW

Adam 结合了 Momentum 和 RMSProp：

- 一阶矩估计 m ：类似 momentum，跟踪梯度均值。
- 二阶矩估计 v ：跟踪梯度平方均值。
- bias correction：修正初始时 m, v 从 0 开始导致的偏差。

常用初始选择：

beta1 = 0.9

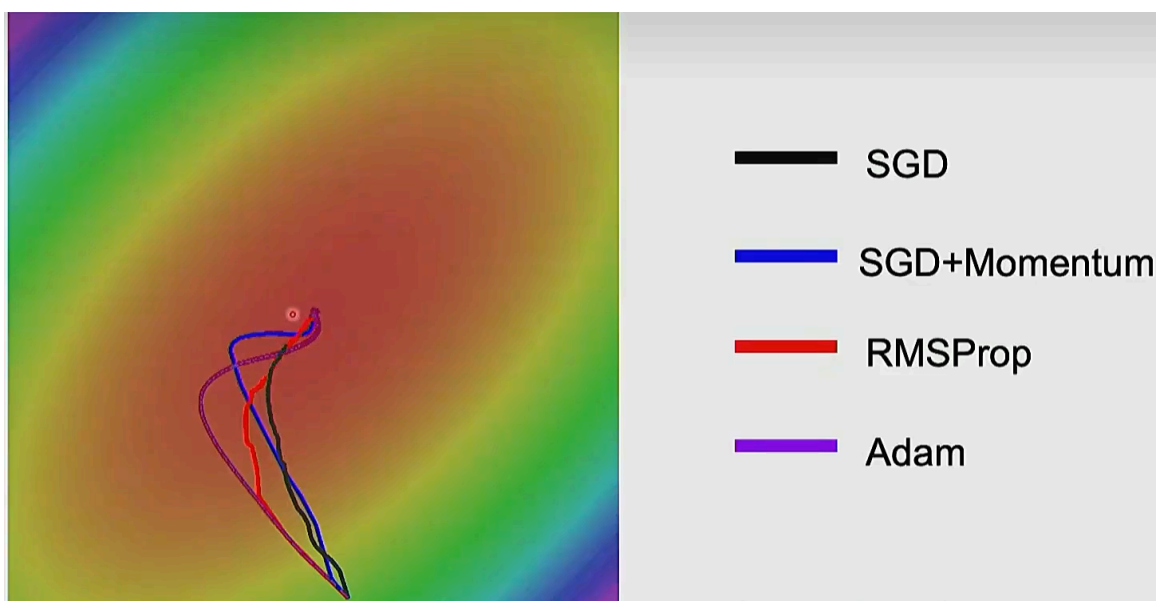
beta2 = 0.999

learning_rate = 1e-3 或 5e-4

AdamW 是 Adam 的常用变体，重点是 **decoupled weight decay**：

- 标准 Adam 中，L2 正则项会进入梯度并参与 moment 计算。
- AdamW 把 weight decay 从梯度 moment 计算中解耦，作为更新后的额外衰减项。

实践上，AdamW 通常是训练深度模型的稳妥默认选择。



7. Learning Rate Schedule

几乎所有优化器都需要学习率。学习率策略决定训练过程中 η 如何变化。

常见策略：

- **Step Decay**: 在固定 epoch 把学习率乘以某个比例，例如 ResNet 常见的 30/60/90 epoch 衰减。
- **Cosine Decay**: 按照余弦曲线平滑下降。
- **Linear Decay**: 线性下降。
- **Inverse Square Root Decay**: 常见于 Transformer 训练。
- **Linear Warmup**: 初期从 0 线性增大学习率，防止训练初期 loss 爆炸。

经验规则：

如果 batch size 增大 N 倍，初始 learning rate 往往也可以近似增大 N 倍。

这只是经验起点，最终仍要靠验证集和训练曲线调参。

一阶优化与二阶优化

First-Order Optimization

只使用梯度构造线性近似：

1. 用梯度近似当前局部损失形状。
2. 沿负梯度方向走一步。

SGD、Momentum、RMSProp、Adam、AdamW 都属于一阶方法。

Second-Order Optimization

二阶方法使用 Hessian 构造二次近似：

$$H = \nabla_W^2 L(W)$$

问题：

- Hessian 有 $O(N^2)$ 个元素。
- 求逆通常需要 $O(N^3)$ 。
- 深度模型参数量可能是千万级或亿级，直接二阶方法不可行。

L-BFGS 等方法会近似 Hessian 或逆 Hessian，但在 mini-batch 噪声环境下效果不总是稳定。

第 4 讲：神经网络与反向传播

1. 从线性分类器到神经网络

线性分类器：

$$s = Wx + b$$

两层全连接神经网络：

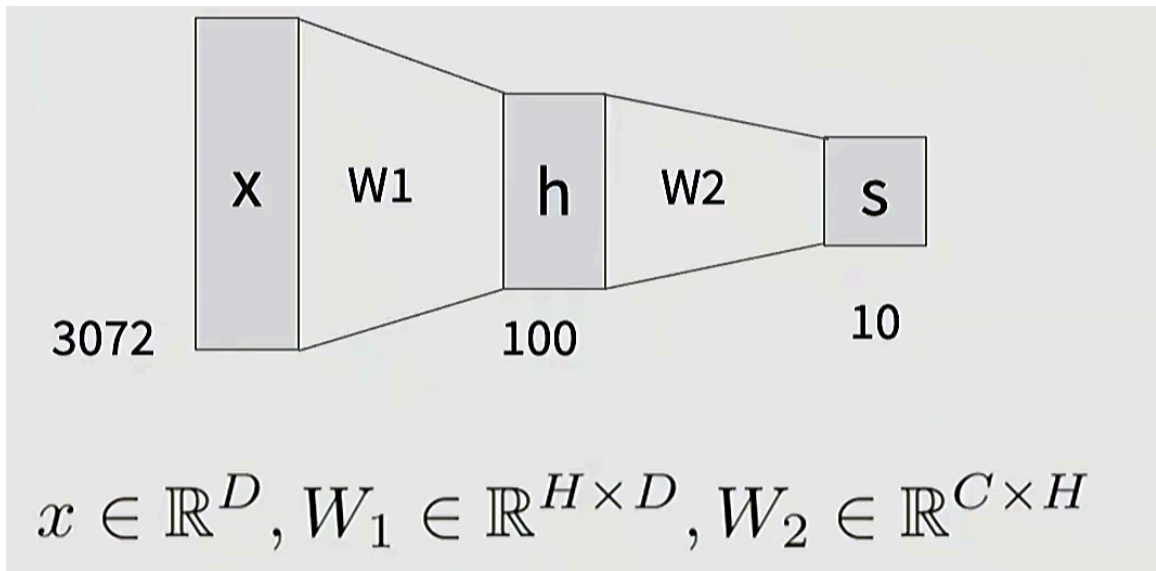
$$h = \sigma(W_1x + b_1)$$

$$s = W_2h + b_2$$

其中：

- x ：输入图像向量。
- h ：隐藏层表示。
- s ：类别分数 logits。
- σ ：activation function，激活函数。

以 CIFAR-10 为例：



- 线性分类器直接学习 10 个类别模板。
 - 两层网络可以先学习大量中间模板，再组合成类别分数。
 - 隐藏层让不同类别可以共享特征。
-

为什么必须有非线性

如果没有激活函数：

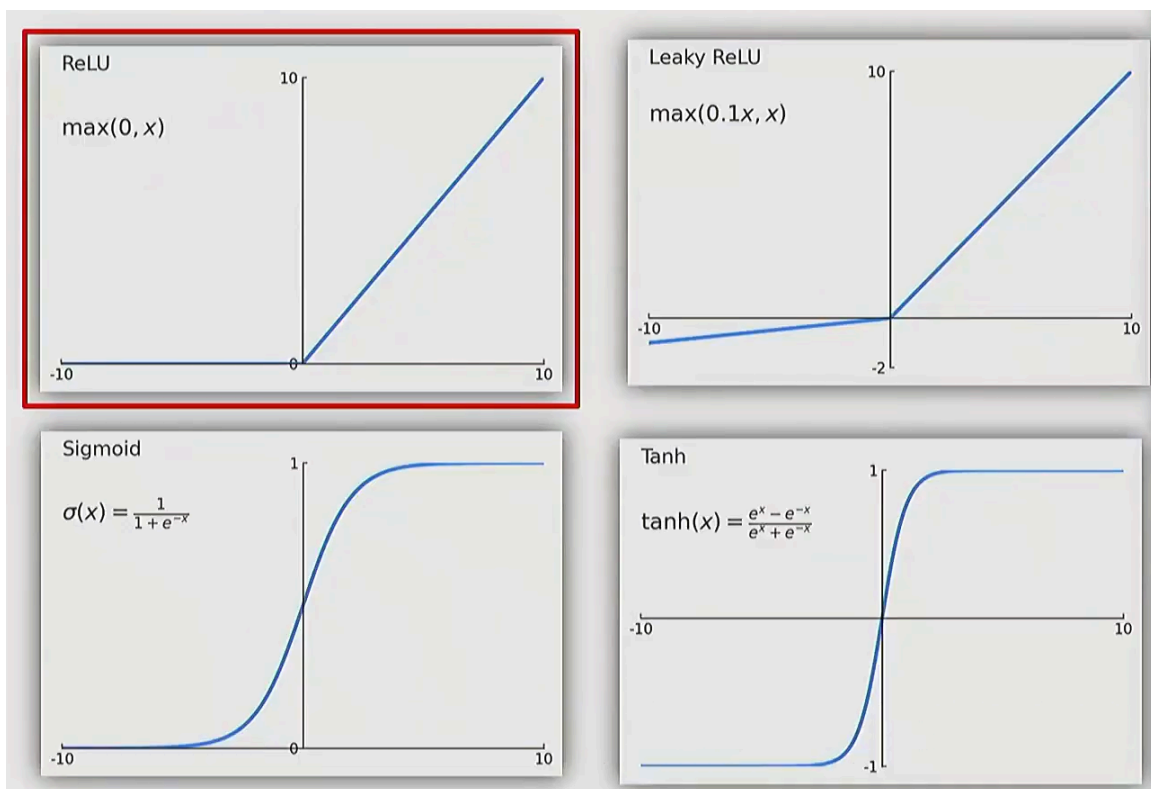
$$s = W_2(W_1x) = (W_2W_1)x$$

这仍然只是一个线性分类器。

非线性的作用：

- 改变输入空间的表示。
- 让原本线性不可分的数据变得可分。
- 多层线性 + 非线性叠加后，模型表达能力远强于单个线性分类器。

常用默认激活函数：



ReLU 的优点:

- 计算简单。
 - 梯度传播相对稳定。
 - 是许多视觉模型的可靠默认选择。
 - 但存在一直为 0 的情况
-

网络层数与命名

常见叫法:

- **2-layer neural network:** 1 个隐藏层 + 1 个输出层。
- **1-hidden-layer neural network:** 同上, 只是强调隐藏层数。
- **3-layer neural network:** 2 个隐藏层 + 1 个输出层。
- **MLP / Fully Connected Network:** 多层全连接网络。

注意:

“Neural Network” 是很宽泛的词; 这里讲的主要是 fully connected network / multi-layer perceptron。

2. 模型容量与正则化

更多层、更宽隐藏层通常意味着更大容量。

但课程强调：

不要主要靠缩小网络规模来做正则化；更常见做法是使用足够大的模型，再用正则化控制泛化。

原因：

- 小模型可能表达能力不足，直接欠拟合。
 - 大模型加合适正则化通常更容易优化，也更容易取得好结果。
-

3. 神经网络和生物神经元的关系

神经网络历史上受到生物神经元启发，但不能过度类比。

差异：

- 生物神经元类型很多，连接模式复杂。
- 树突本身可能执行复杂非线性计算。
- 突触不是单个静态权重，而是复杂动态系统。
- 人工神经网络为了计算效率，通常组织成规则层结构。

所以更准确的理解是：

现代神经网络是可微分函数组合系统，而不是大脑的精确模拟。

4. 神经网络的训练目标

把线性打分函数替换为非线性网络后，整体训练目标仍然是：

$$L = L_{\text{data}} + \lambda R(W)$$

例如：

输入图像 X

```
-> 神经网络产生 logits  $s$   
-> Softmax 或 SVM loss  
-> 加上 regularization  
-> 得到总损失  $L$ 
```

关键问题变成：

如何高效计算每个参数对损失的梯度？

也就是：

$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial b_2}$$

手推完整矩阵微积分对复杂模型不可行，因此需要计算图和反向传播。

5. Computational Graph

计算图把复杂函数拆成简单操作节点：

$x, W \rightarrow \text{multiply} \rightarrow \text{scores} \rightarrow \text{loss} \rightarrow L$

每个节点负责：

- forward: 计算输出，并缓存 backward 需要的中间值。
- backward: 根据 upstream gradient 和 local gradient 计算 downstream gradients。

这种模块化方式使复杂模型可组合：

- 换 loss 不需要重推整个模型。
 - 换层结构只要每个节点实现 forward/backward。
 - PyTorch、TensorFlow 等框架本质上都在维护类似的计算图和自动求导系统。
-

6. Backpropagation 的核心

反向传播就是在计算图上递归应用链式法则。

对一个节点：

输入 $x \rightarrow$ 节点 $f \rightarrow$ 输出 $z \rightarrow$ 损失 L

如果已知 upstream gradient:

$$\frac{\partial L}{\partial z}$$

并能计算 local gradient:

$$\frac{\partial z}{\partial x}$$

那么：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}$$

一句话：

downstream gradient = upstream gradient $\times$ local gradient

7. 常见计算节点的梯度模式

Add Gate

$$z = x + y$$

加法节点把 upstream gradient 原样分发给两个输入：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}$$

$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}$$

Multiply Gate

$$z = xy$$

乘法节点会交换乘数：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot y$$

$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \cdot x$$

Copy Gate

如果一个变量被多个后续节点使用，反向传播时梯度相加：

$$\frac{\partial L}{\partial x} = \sum_k \frac{\partial L^{(k)}}{\partial x}$$

Max Gate

$$z = \max(x, y)$$

梯度只传给 forward 中取到最大值的输入，另一个输入梯度为 0。

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

局部梯度：

$$\frac{d\sigma}{dx} = \sigma(x)(1 - \sigma(x))$$

8. Backprop 实现：forward / backward API

每个模块通常有两个接口：

```

forward(x):
    计算输出 out
    缓存 backward 需要的中间值
    返回 out

backward(dout):
    使用缓存和 upstream gradient dout
    计算 dx, dW, db 等梯度
    返回梯度

```

forward pass:

- 从输入到输出计算分数和损失。
- 保存中间结果。

backward pass:

- 从损失开始反向传播。
- 每个节点用链式法则计算输入和参数的梯度。

9. 向量、矩阵和张量的反向传播

标量情形下，梯度是数字；向量和矩阵情形下：

- x 是向量时， $\frac{\partial L}{\partial x}$ 与 x 形状相同。
- W 是矩阵时， $\frac{\partial L}{\partial W}$ 与 W 形状相同。
- 损失 L 仍然是标量。

理论上向量到向量的导数是 Jacobian：

$$J = \frac{\partial z}{\partial x}$$

但实际深度学习中几乎不会显式构造完整 Jacobian，因为内存和计算量太大。

实践做法：

隐式计算 Jacobian-vector product，只保留需要的梯度张量。

10. ReLU 的向量反传

逐元素 ReLU：

$$z = \max(0, x)$$

反向传播：

$$\frac{\partial L}{\partial x_i} = \begin{cases} \frac{\partial L}{\partial z_i}, & x_i > 0 \\ 0, & x_i \leq 0 \end{cases}$$

- forward 中激活的神经元传递梯度。
 - 被截断为 0 的神经元不传递梯度。
-